D6CIM: 60.4-TOPS/W All-Digital 6T-SRAM-Based Compute-in-Memory Macro Supporting 1-to-8 b Fixed-Point Arithmetic in a 28-nm CMOS

Jonghyun Oh, *Member, IEEE*, Chuan-Tung Lin, *Member, IEEE*, and Mingoo Seok, *Senior Member, IEEE*

***Abstract*—This article presents an all-digital 6T-SRAM-based compute-in-memory (D6CIM) macro that supports 1-to-8-bit fixed-point vector matrix multiplication. The D6CIM is fully digital, employing only digital standard cells and 6T SRAM bitcells. The D6CIM adopts a time-sharing architecture with an optimal degree of time-sharing to maximize the product of compute density and weight density. It also features several circuit techniques, namely non-precharge single-ended access, hybrid compressor adder-tree circuits, and bidirectional shift-accumulation. We prototype the D6CIM macro in a 28-nm CMOS technology. The maximum operating frequency is ~360 MHz in a 1.1-V supply. The energy efficiency is measured at up to 60.4 TOPS/W. It achieves a compute density of 1.46 TOPS/mm^2 and a weight density of 1005 kb/mm^2. Compared to prior state-of-the-art CIM macros, the D6CIM achieves 1.3-4.8X improvement in the product of energy efficiency, compute density, and weight density.**

***Index Terms*—Compute-in-memory (CIM), in-memory computing (IMC), digital compute-in-memory (DCIM), time-sharing, hardware reuse, non-precharge single-ended access, hybrid compressor adder-tree, bidirectional shift-accumulator.**

I. Introduction

OVER the last several decades, the deep neural network (DNN)-based artificial intelligence (AI) has made significant progress. However, progress comes with rapidly increasing computational complexity, and as a result, modern DNN models demand faster, more efficient computing systems. Responding to this demand, the computing hardware research community has proposed several SRAM-based compute-in-memory (CIM) macros. CIM (computing-in-memory) macros can perform vector-matrix multiplication (VMM) without row-by-row memory access. As a result, they have largely improved the performance and energy efficiency.

Most of the earlier works proposed analog-mixed-signal CIM (ACIM) macros [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14]. The ACIM macros perform VMM using analog computing mechanisms such as current summing [14], charge coupling [9], [12], and charge sharing [2]. Those ACIM macros achieve high performance and energy efficiency by leveraging the compact analog circuits. However, they require analog-to-digital and digital-to-analog conversion (ADC, DAC) at their inputs and outputs, which limits achievable performance and energy efficiency. In addition, the analog circuits exhibit significant variability across process, voltage, and temperature (PVT) variations, resulting in non-negligible errors in the computation results.

To address those limitations, recent works have proposed the digital version of the CIM macros, namely DCIM [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33]. Indeed, DCIM macros do not require data conversion and are therefore free of the associated overhead. Additionally, as they are based on robust digital logic circuits, the DCIM macros produce full-precision output without error across PVT variations. However, digital circuits are significantly bulkier than their analog counterparts, resulting in degraded weight density and compute density.

Recent works have explored reducing the area overhead of DCIM macros. Some of the works propose to employ approximate arithmetic hardware. Lin et al., for example, proposed replacing some full adder cells with NAND and NOR cells, which reduces silicon area at the cost of increased errors [25]. On the other hand, Yan et al. proposed to share arithmetic hardware across inputs over time [15]. Such a hardware-reuse architecture can effectively reduce silicon area; however, it tends to decrease computing throughput.

In this work, we propose a novel digital 6T-SRAM-based CIM macro, titled D6CIM [34], featuring a time-sharing architecture where the arithmetic hardware is shared across different data (weights and inputs). We start the design of D6CIM with the model-based architecture optimizations. Specifically, we strive to determine the optimal level of hardware reuse that maximizes the product of a compute density and a weight density. The optimal level of hardware reuse found guides us in determining (i) the number of bitcells sharing one multiplier and (ii) the bit-serial vs. bit-parallel scheme for inputs. In addition, we develop several circuit-level techniques, including a non-precharge single-ended access, a hybrid compressor adder-tree (HCA), and a bidirectional shift-accumulator (BSA), to further enhance the performance and efficiency of the proposed DCIM macro. The rest of this article is organized as follows. Sec. II describes the model-based design parameter exploration. Sec. III describes the proposed architecture of the D6CIM, and the proposed circuit techniques. Sec. IV presents the D6CIM prototype and

Received 4 December 2025; revised 25 January 2026; accepted 15 February 2026. Date of publication 24 February 2026; date of current version 1 July 2026. This work was supported in part by COGNISENSE, one of the seven centers in Joint University Microelectronics Program 2.0 (JUMP 2.0), an SRC Program sponsored by Defense Advanced Research Projects Agency (DARPA); in part by Semiconductor Research Corporation (SRC) TxACE (Task 3160.002); and in part by the Defense Advanced Research Projects Agency (DARPA) under Contract HR001121C0134. This article was recommended by Associate Editor D. John. *(Corresponding author: Mingoo Seok.)*

The authors are with the Department of Electrical Engineering, Columbia University, New York, NY 10027 USA (e-mail: ms4415@columbia.edu).

Digital Object Identifier 10.1109/TCSI.2026.3665839

1549-8328 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:27:34 UTC from IEEE Xplore. Restrictions apply.

measurement results. Sec. V concludes the paper. Finally, we also provide an appendix that details the model derivation processes.

II. Model-Based Design-Parameter Exploration

We aim to systematically determine the optimal degree of hardware sharing for DCIM. Some prior works have presented time-sharing architectures that significantly improve weight density [15], [17], [18]. However, they lack systematic optimizations. Rather, they often presented single design points without providing the rationale for hardware design choices. In this work, we develop several simple models of generic DCIM hardware and, using these models, explore the design parameters. The exploration enables us to determine two key design parameters: i) the number of bitcells sharing one multiplier ($N_{cell/mult}$) and ii) the bit-serial vs bit-parallel scheme for inputs.

We start the model-based exploration process by formulating a simple model of the weight density (d_{weight}) as follows:

$$d_{weight} = \frac{k_W \cdot A_{mem}}{A_{mem} + A_{ari}}, \tag{1}$$

where A_{mem} is the silicon area of the bitcell array and A_{ari} is that of the arithmetic hardware. k_W roughly represents the theoretical maximum weight density of a macro in the unit of Kb/mm^2. We ignore the peripheral's area (A_{peri}) and approximate the CIM macro's area (A_{macro}) as a sum of A_{mem} and A_{ari}. Note that it is not possible to achieve the theoretical maximum weight density of k_W since to do so, A_{ari} must be 0, and the macro at that point is no longer CIM.

Similarly, we formulate the compute density (d_{comp}) as:

$$d_{comp} = \frac{k_C \cdot A_{ari}}{A_{mem} + A_{ari}}, \tag{2}$$

where k_C represents roughly the theoretical maximum compute density of a macro. Its unit is tera operations per second per millimeter square (TOPS/mm^2). Again, it is not possible to achieve the theoretical maximum compute density of k_C since to do so, A_{mem} must be 0, and at that point, the macro no longer works as memory hardware.

By combining (1) and (2), we can derive:

$$\frac{d_{weight}}{k_W} + \frac{d_{comp}}{k_C} = 1, \tag{3}$$

which shows that the compute density and the weight density are in a trade-off relationship. As shown in Fig. 1, increasing d_{weight} would linearly decrease d_{comp} and vice versa. Additionally, it demonstrates that increasing k_W and k_C can shift the trade-off line upward, thereby improving the achievable bounds of d_{weight} and d_{comp} for a CIM macro.

In this design parameter exploration, we choose to maximize the product of the compute density and the weight density. We did not include energy efficiency in the goal, as the time-sharing architecture, for the first order, does not improve energy efficiency. Indeed, the number of toggles, hence dynamic energy consumption, roughly remains the same across different degrees of hardware sharing. Although parameters such as A_{mem} and A_{ari} can affect the lengths of critical wires, we choose to ignore these secondary effects for the sake of a simpler analysis. Note that some prior work considers the product of energy efficiency and compute density (without weight density) as a metric [21]. However, we did not consider it since we can artificially improve this metric by simply adding more arithmetic hardware.

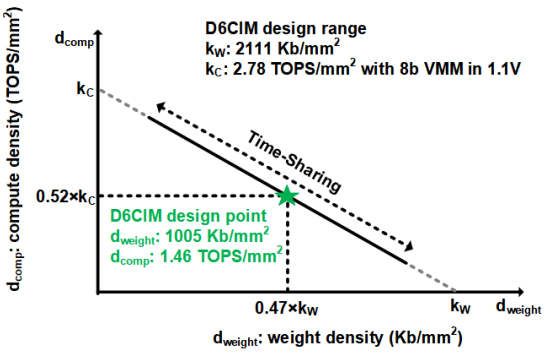


Fig. 1. The time-sharing architecture faces a tradeoff between weight density and compute density. The design point adopted by D6CIM is marked.

We can formulate the product of the compute density and the weight density as follows:

$$d_{weight} \cdot d_{comp} = k_W \cdot k_C \cdot \frac{A_{mem} \cdot A_{ari}}{(A_{mem} + A_{ari})^2} \tag{4}$$

By solving the partial derivatives of (4) equal to 0, we can find that the product would be maximized if A_{mem} is equal to A_{ari} for a given total area ($A_{mem} + A_{ari}$).

This finding leads us to rule out the use of non-6T bitcells, such as the 8T or 10T bitcell with an embedded multiplier [22], [25]. Those non-6T bitcells increase A_{ari} significantly beyond the optimal level. This is because those embedded multipliers already consume 33% to 66% of the bitcell area. Furthermore, they demand to add a substantial amount of arithmetic hardware, such as adder trees and accumulators, to consume all of the outputs of the multipliers in a timely manner.

For the same reason, we also choose the bit-serial input feeding scheme to manage A_{ari} at the optimal level. Indeed, the bit-serial scheme allows us to employ simple 1-bit multipliers in sub-columns. If we adopt the bit-parallel scheme, e.g., an 8-bit-parallel one, we need to employ 8-bit×1-bit multipliers in sub-columns, largely increasing A_{ari}. Also, they will produce partial products eight times as large. To consume them on time, we must proportionally increase the size of the following arithmetic hardware (e.g., adder trees), again increasing A_{ari} beyond the optimal level.

Therefore, we decided to stick to the 6T bitcells and the bit-serial input scheme. Then, we evaluate a range of time-sharing architectures to find one that has A_{ari} similar to A_{mem}. We do so by developing parameterized models of A_{ari}, A_{mem}, and so on. We supplement the model derivation process in the appendix (Sec. A). Fig. 2 shows the results of this model-based evaluation. N_{subcol} is the number of sub-columns, each of which contains two multipliers, in a bitcell column and is formulated to:

$$N_{subcol} = N_{rows} \div (2 \times N_{cell/mult}), \tag{5}$$

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:27:34 UTC from IEEE Xplore. Restrictions apply.

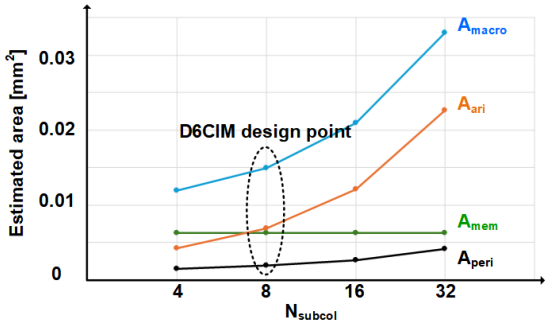


Fig. 2. Model-based estimation of A_{macro}, A_{mem}, A_{ari}, and A_{peri} across N_{subcol} from 4 to 32. D6CIM adopts N_{subcol} of 8.

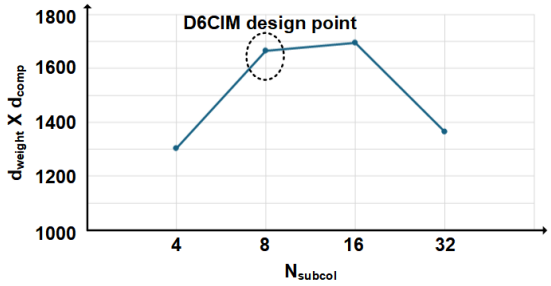


Fig. 3. Model-based estimation of the product of d_{weight} and d_{comp} across N_{subcol}. D6CIM adopts N_{subcol} of 8 to achieve a better d_{weight}. The difference of the product of d_{weight} and d_{comp} is only 1.6% between N_{subcol} of 8 and 16.

where N_{row} is the total number of rows of the bitcell array. The results show that sharing two multipliers across 16 bitcells (i.e., N_{subcol} = 8 or $N_{cell/mult}$ = 8 for N_{rows} = 128) can make A_{ari} close to the optimal level.

Then, we investigate if the chosen N_{subcol} value actually maximizes the product of d_{weight} and d_{comp}. Again, to do so, we develop the parameterized models of d_{weight} and d_{comp}. We will present the model derivation in the appendix (Sec. A) for the interested readers. Fig. 3 summarizes the results, which show that the product is maximized when N_{subcol} is 16. Instead, we choose N_{subcol} to be 8. This is because we aim to achieve a state-of-the-art weight density of 1000 kb/mm^2. The difference of the product of d_{weight} and d_{comp} is only 1.6% between N_{subcol} of 8 and 16. The final macro design achieves A_{mem} of 41.6% and A_{ari} of 45.8% of the macro area, closely aligning with the target.

The design parameter exploration helps us balance weight density and compute density, thereby maximizing their product. Without this design-parameter analysis, we could end up with a sub-optimal design. For example, if we determine to employ a single multiplier per bitcell (i.e., $N_{cell/mult}$ = 1), the resulting macro may achieve very high compute density but poor weight density. In addition, to improve energy efficiency, we developed the circuit-level techniques, such as the non-precharge-based access (Sec. III-B), the hybrid compressor adder-tree (Sec. III-C), and the bidirectional shift-accumulator (Sec. III-E), which help to reduce switching activities and transistor counts.

Note that we streamline an analysis by not including the layout effects in the modeling process. Including layout effects will improve the model's accuracy, but it will greatly increase its complexity. Also, the goal of the modeling in our study is to compare different architectures rather than to accurately estimate energy consumption and throughput. We believe the transistor count-based model is sufficient for our goal.

III. D6CIM Architecture and Circuits

A. Architecture

Fig. 4 shows an overall architecture of the D6CIM. It has a total of 128 × 128.6T SRAM array to store weight values. The D6CIM supports 8-bit weights, and a single macro can store 128×16 weights. The macro dimensions and bitwidth are determined based on the target neural network models and the accelerator architecture. Specifically, we choose the 128 × 128 dimensions for D6CIM because we want to use 16 of them in an accelerator to map ResNetV1, DSCNN, and MobileNetV1 [26], [35].

The basic building block of the macro is a *sub-column*, which has 16 bitcells and two 1-bit multipliers (NOR gates), connected to a local bitline (LBL) and a local bitline bar (LBLb). We choose to have 16 bitcells per sub-column based on our model-based parameter exploration (Sec. II). Eight sub-columns make one column, where those sub-columns share a global bitline (GBL) and a global bitline bar (GBLb). Eight columns share one HCA and one BSA.

The macro also contains the row and column peripherals. The row peripherals control 16 input-vector lines (ILs). Two ILs supply input bits to the two 1-bit multipliers in the sub-columns in the same row. The row peripherals also control 128 · 2 wordlines (WLs). Note that we employ the split WLs [36], which independently control the two access transistors of a bitcell. On the other hand, the column peripherals control the GBLs and GBLbs.

The D6CIM can perform one 8-bit 128 × 16-d(imension) VMM in 64 clock cycles. Fig. 5 shows the waveforms of an exemplary operation. In the first cycle, the row peripherals activate two consecutive WL1 in each sub-column (e.g., WL1[0] and WL1[1] in the sub-column0 in Fig. 4), which reads out two weight bits to the NOR gates via LBL and LBLb. At the same time, the row peripherals feed the first bits of 16 8-bit input elements via 16 ILs, two bits of which are fed to the sub-columns in the same row. Taking those weight bits and input bits, the NOR gates produce the partial products (e.g., PP[0], [0], PP[1], [0] in the sub-column0). All 64 sub-columns of the D6CIM macro perform the same operation in the same cycle, resulting in producing 16 8-bit partial products, i.e., PP[15:0][7:0], per every eight columns. The HCA, shared by every eight columns, sums the 16 8-bit partial products into one 13-bit result. Then, the BSA, also shared by every eight columns, accumulates the HCA's outputs and produces the BSA's output (e.g., VMM_{out}[0] for Column0 to Column7).

In cycles 2 to 8, the macro repeats the same process while feeding the next bits of the same 16 8-bit input elements. The BSA accumulates 8 13-bit results. Then, for cycles 9 to 16, the macro repeats the same process while working on the next 16 elements of the input. This process repeats until the macro feeds all 128 elements of the input vector over 64 cycles. During this process, the BSA accumulates the HCA's output

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:27:34 UTC from IEEE Xplore. Restrictions apply.

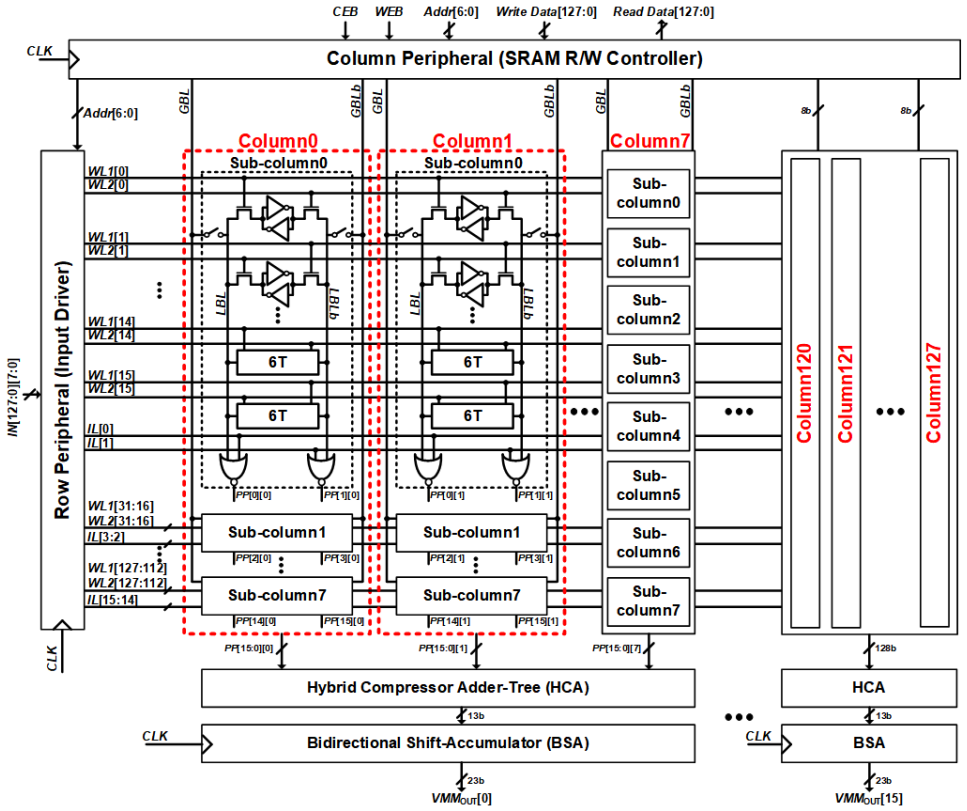


Fig. 4. Proposed D6CIM architecture.

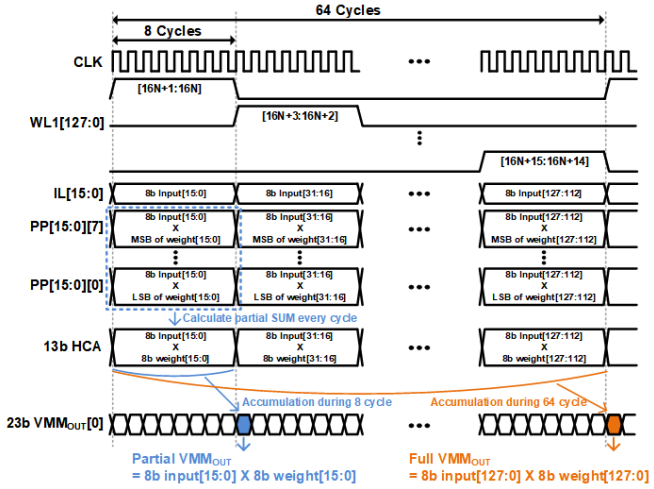


Fig. 5. Operational waveforms of the D6CIM when it performs an 8-bit in-memory computing operation over 64 cycles.

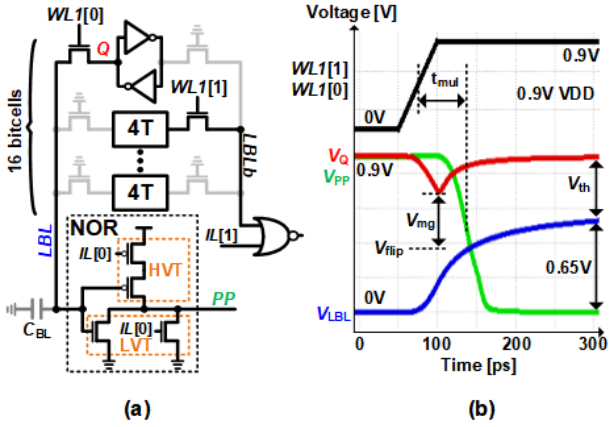


Fig. 6. (a) Schematic of the non-precharge/static single-ended access scheme and (b) simulated transient operation.

and ultimately produces the 23-bit full-precision output. Fig. 5 summarizes the dataflow above. It takes eight cycles to feed 16 8-bit input elements serially, and it repeats this process eight times to access all 16 bitcells in each sub-column. The macro takes 64 clock cycles to complete one 8-bit VMM operation.

B. Non-Precharge/Static Single-Ended Access for Computing

Specifically, for in-memory computing, we develop a non-precharge (i.e., static) single-ended technique to access bitcells. The proposed non-precharge technique does *not* precharge bitlines to save power consumption. It is different from the technique in [15] that uses precharge for VMM. Figs. 6(a) and (b) show how the macro accesses the bitcells for computing. First, the macro does *not* precharge LBL and LBLb. In the example, V_Q is initially 0.9V and V_{LBL} is 0V. Then, it asserts WL1[0] and WL1[1] simultaneously. WL1[0] enables the left access transistor of the first bitcell, while WL1[1] enables the right access transistor of the second bitcell. As WL1[0] rises, the bitcell starts charging LBL, which drives the NOR gate (multiplier).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:27:34 UTC from IEEE Xplore. Restrictions apply.

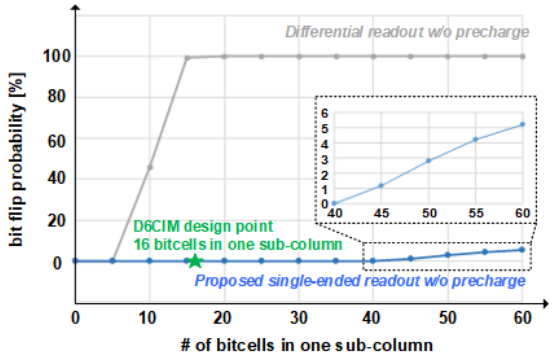


Fig. 7. The proposed non-precharge/static single-ended access technique attains a lower bit flip probability than a differential counterpart. D6CIM's design point is set to 16 bitcells per sub-column to ensure high robustness. Based on 1000-sample Monte-Carlo simulations at a 0.9-V supply voltage.

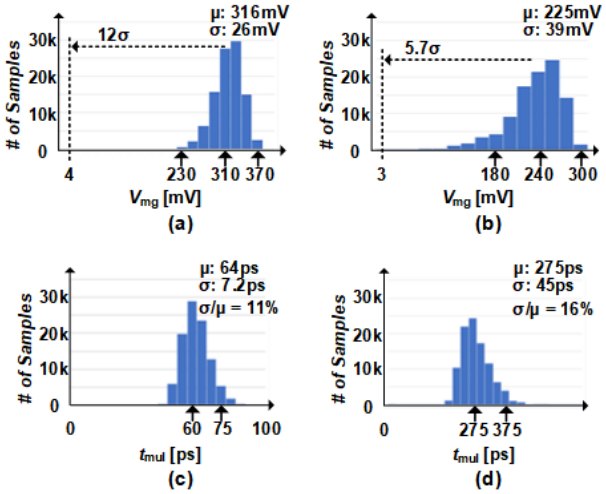


Fig. 8. Monte-Carlo simulation for (a) V_{mg} at 0.9V and (b) V_{mg} at 0.6V, and for (c) t_{mul} at 0.9V and (d) t_{mul} at 0.6V.

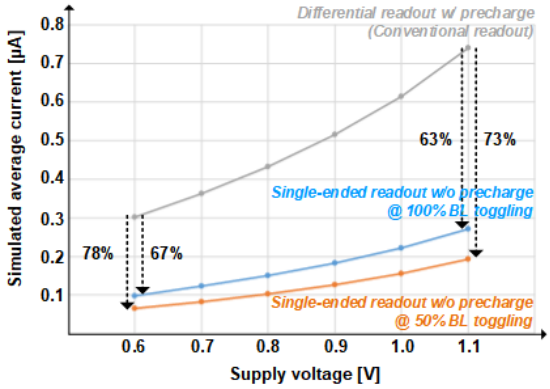


Fig. 9. The proposed non-precharge/static single-ended access technique reduces the current draw as compared to the conventional precharge-based differential counterpart. Based on the simulations of one sub-column containing 16 bitcells.

Note that, as shown in Fig. 6(b), the lack of precharge operation induces a voltage drop in V_Q due to the charge sharing between the bitcell storage node (Q) and LBL. If this drop gets severe, it can flip the content of the bitcell, e.g., V_Q flips to 0V in this example. Due to this risk, indeed, the conventional SRAM design does not employ such a non-precharge access technique.

We address the read disturbance problem of the proposed static single-ended access technique via two methods. First, we limit the total number of bitcells that share the LBL and LBLb to 16. The shorter bitlines have less parasitic capacitance, significantly reducing the chance of bit flipping. Second, we find that single-ended access, which we initially chose to improve computing throughput, also improves robustness against read disturbances. This is because one of the bitcell storage nodes remains undisturbed.

We conducted a simulation-based study to verify the robustness of the proposed design. Fig. 7 shows the bit flip probability of the proposed technique vs a static differential read scheme as a function of the number of bitcells in one sub-column. It shows that the static differential read scheme quickly loses robustness as the number of bitcells exceeds five. However, the proposed technique retains 0% until it reaches 40. Based on the results, we conservatively set the number of bitcells in a sub-column to 16.

We further verify the robustness of the proposed access technique at the chosen design point (i.e., 16 bitcells) using Monte Carlo simulations. To do so, we define a metric called voltage margin V_{mg}, which can be formulated as:

$$V_{mg} = min(V_Q - V_{flip}), \tag{6}$$

where V_{flip} is the level where the bitcell flips. We can suppose the bitcell is robust if V_{mg} is positive. The 100k Monte-Carlo iterations considering local transistor mismatch and global process corners show that the mean and standard deviation of V_{mg} are 316 mV and 26 mV, respectively, confirming V_{mg} can remain positive even in the ~12-σ worst-case condition (Fig. 8(a)). We repeat the same simulation at 0.6V, confirming V_{mg} remains positive even in the ~5.7-σ worst-case condition (Fig. 8(b)).

The proposed static/non-precharge single-ended readout technique also improves energy efficiency. Fig. 9 shows the simulation results, where the proposed technique draws 63-78% less current than the conventional dynamic differential readout counterpart across 0.6-1.1V, due to the absence of precharge operation.

On the other hand, Fig. 6(b) shows that the potential of LBL (V_{LBL}) is not rail-to-rail. It settles to VDD-V_{th}, where V_{th} is the threshold voltage of the access transistor. Still, to robustly sense this low voltage, we use a high-V_{th} PMOS and a low-V_{th} NMOS in the NOR gate to lower the NOR gate's switching point.

We evaluate the robustness of the NOR gate using Monte Carlo simulations. We simulate the latency from WL1[0] to the output of the NOR gate, which we define as t_{mul}. Fig. 8(c) shows the simulation results at 0.9V. The delay shows a mean of 64 ps and a standard deviation of 7.2 ps. The 3-σ worst-case t_{mul} takes only <2% of the clock period (The D6CIM operates with 230 MHz in 0.9V). We also confirm similar variability at 0.6V (Fig. 8(d)). Note that the computational results are

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:27:34 UTC from IEEE Xplore. Restrictions apply.

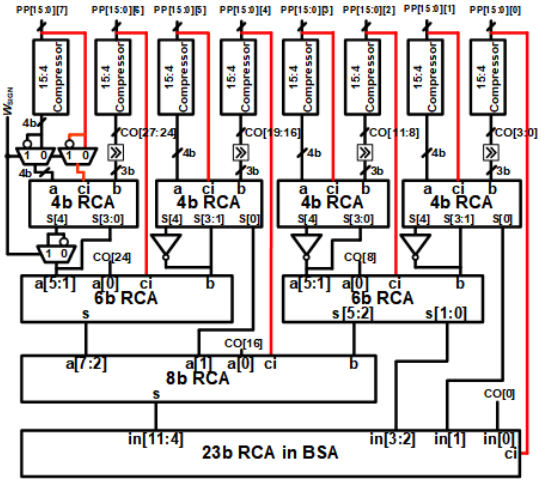


Fig. 10. The proposed HCA schematics.

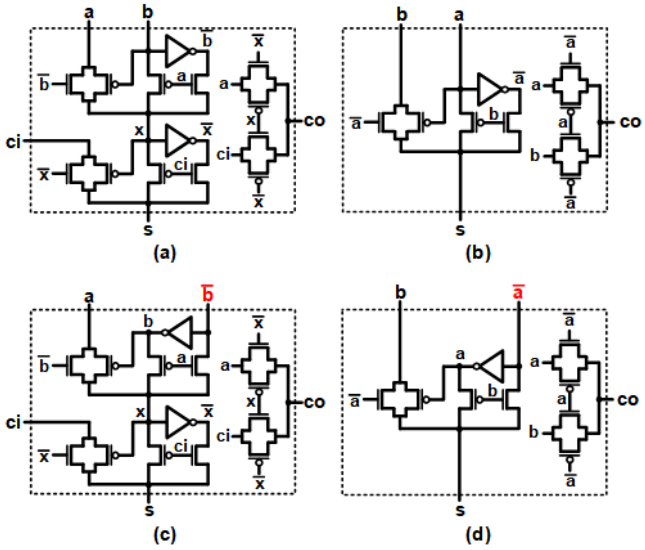


Fig. 11. Schematic of (a) 16T transmission-gate (TG)-based FA and (b) 10T TG-based HA; and (c) one-input-inverted FA and (d) HA.

generally robust to such delay variations, since D6CIM is fully digital.

C. Hybrid Compressor Adder-Tree

The NOR-gate multipliers in the sub-columns produce a large number of partial products. To sum up all the partial products, instead of using a single large multi-input adder tree, we design a hybrid compressor adder tree (HCA). Fig. 10 shows the detailed schematic of the HCA. It consists of (i) eight 15:4 compressors and (ii) one 8-input, 4-bit adder tree. Note that the HCA must take a 128-bit input. However, the eight compressors can only accept a 120-bit input. We could add one more compressor; however, it increases the area overhead. Therefore, we retain the eight compressors, but route the remaining eight bits of the input to the carry-in port of the HCA's adder tree and the BSA's ripple-carry adder (RCA). Note that the HCA supports the signed 2's complement arithmetic by conditionally inverting some of the most-significant bits (MSB).

We also optimize the 8-input 4-bit adder tree in the HCA. To do so, we develop a transmission gate (TG)-based 16T FA cell (Fig. 11). The TG-based FA cell can reduce transistor counts compared to a CMOS-based 28T FA cell, albeit at the cost of a moderately increased delay [23]. We insert a buffer/inverter in every four FA cells to maintain the driving strength. We conducted a simulation-based study to find the trade-off between delay and buffering overhead. Fig. 12 shows the results, where inserting a buffer in every four FA cells strikes a good balance between delay and transistor count. In addition, to further conserve space, we insert an inverter, not a buffer. However, the inserted inverter inverts the polarity. To reverse the inverted polarity with minimal hardware, we also develop one-input-inverted FAs. Note that we use the transistor count as a proxy for the silicon area. It would be more accurate to use the layout area. Unfortunately, creating the layout for each design is prohibitively time-consuming, so we use the transistor count as an estimate.

We also investigate the robustness of the TG-based cells against PVT variation. To do so, we perform a comparative

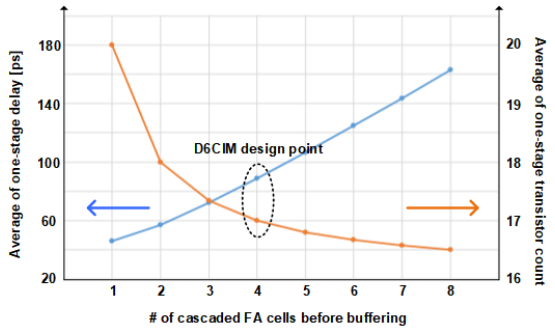


Fig. 12. Simulated delay of the cascaded TG-based FAs in 0.9-V supply. D6CIM adopts a buffer every four stages of FA cells.

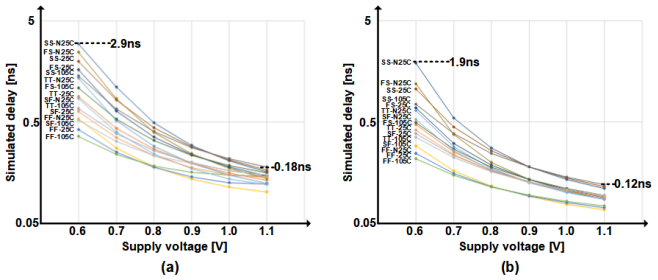


Fig. 13. The delay comparisons of (a) the TG-based FA and (b) CMOS FA cells. The four cascaded FAs, followed by an inverter, are considered over PVT corners.

study between the proposed TG-based FA cell with the CMOS counterpart. We consider four cascaded FAs, followed by one inverter. Fig. 13 shows the results. We found that both gates are robust, while the TG-based gates exhibit approximately 50% longer delays across supply voltages and corners. Overall, as shown in Fig. 14, the proposed HCA architecture reduces the transistor count by ~15% as compared to employing only one larger adder tree [23]. Additionally, the proposed TG-based

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:27:34 UTC from IEEE Xplore. Restrictions apply.

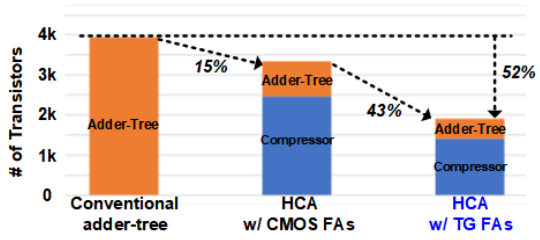


Fig. 14. The HCA and TG-based FA saves a total of 52% transistor counts as compared to the conventional one large multi-input adder tree.

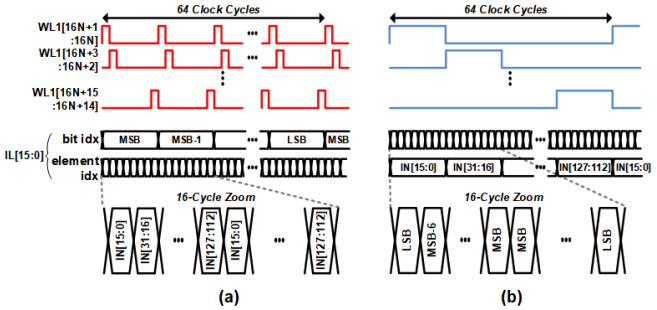


Fig. 15. (a) Element-wise input feeding fashion exhibiting large WL switching activities and (b) the proposed bit-wise feeding fashion showing reduced WL switching.

FA cells enable an additional 43% reduction in the transistor count in the HCA.

D. Bit-Wise Input Feeding Scheme

The D6CIM has access to only 2 of the 16 weights in a given cycle. Therefore, we should feed only two out of every 16 elements in the input vector. It can be sub-optimal if we just feed all elements in the input vector, since each sub-column can use only two out of every 16 weights in each cycle. We also employ the bit-serial input feeding scheme, following the practices in many of the prior works [15], [17], [23]. The primary reason for employing the bit-serial scheme is that it requires simple 1-bit multipliers in the sub-columns, as we elaborated on this decision in Sec. II.

With those two design decisions, now we have another question: after feeding the first two input bits, e.g., IN [7], [15] and IN [7], [14], should we feed the next input elements, e.g., IN [7], [13] and IN [7], [12], or should we feed the next bits of the same input elements, e.g., IN [6], [15], and IN [6], [14]? We call the former *element*-wise and the latter *bit*-wise input feeding scheme, and it is quite evident that the bit-wise scheme is more energy efficient. This is because the bit-wise scheme induces fewer switching activities in WLs, thereby reducing the toggling count of LBLs and LBLbs. As shown in Fig. 15, it reduces the WL switching count by 8X, while maintaining a similar activity level in ILs compared to the other scheme.

E. Bidirectional Shift-Accumulation

We propose a bidirectional shift-accumulation scheme. Most, if not all, prior works employ a unidirectional shift-accumulation approach [25]. The key idea of the proposed bidirectional scheme is that we reverse the bit-serial direction

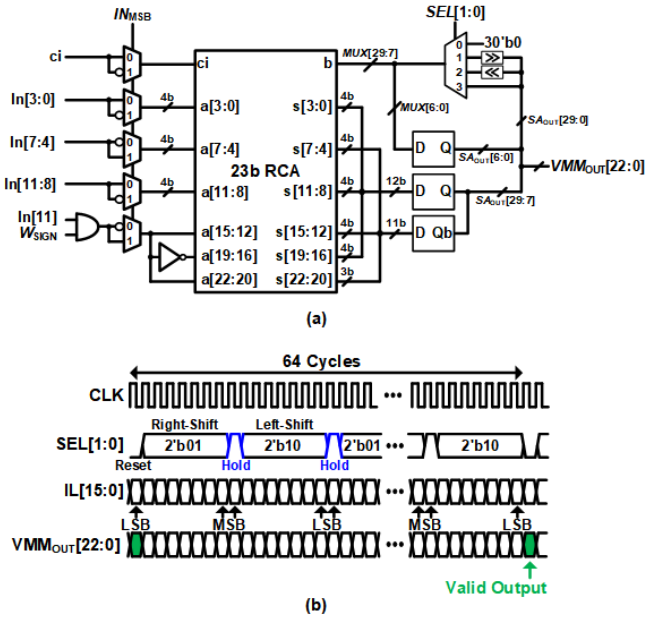


Fig. 16. (a) Schematic of the proposed BSA design and (b) timing diagram of the BSA operation with 8-bit input vector.

when the macro completes the computation of the current input elements and starts the next input elements. For example, if we feed the first input element from MSB to LSB, we feed the second from LSB to MSB. Fig. 16(a) shows the schematics of the proposed bidirectional shift-accumulator (BSA). It comprises a 23-bit RCA, a 30-bit register, a bi-directional shifter, and several MUXes. The 4:1 MUX can select the accumulator output SA_{out}, its left-shifted value, its right-shifted value, or 0, and feeds the selected one to the *b* port of the RCA. Fig. 16(b) shows the operation. The first seven cycles select the right-shifted values; the eighth cycle selects the non-shifted value; the next seven cycles select the left-shifted values. This pattern repeats during 64 clock cycles.

Note that the additional MUX increases the delay of the backward (feedback) path of the BSA. However, the critical path of the macro during vector-matrix multiplication is the forward path from the bitcells to the HCA and then to the BSA. Hence, the additional MUX delay does not affect the macro's critical delay.

The proposed bidirectional shift accumulator demands less hardware: one bidirectional shifter, one 0-bit shifter (wires), and one MUX. We also need only minimal modifications to the row peripherals. This scheme can also support variable input precision, ranging from 1 to 8 bits, with minimal change. On the other hand, the conventional unidirectional shift-accumulator requires much larger hardware. This is because the unidirectional shift-accumulator must shift *back* to the initial bit position during the input element transition. To support 1-8 bit precision, for example, the conventional shift-accumulator must include eight backward shifters, one forward shifter, and a MUX, which incurs significant area overhead.

F. Layout Optimization

Our primary goal during the physical design is to minimize critical wires, such as LBLs, WLs, and ILs. This is because

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:27:34 UTC from IEEE Xplore. Restrictions apply.

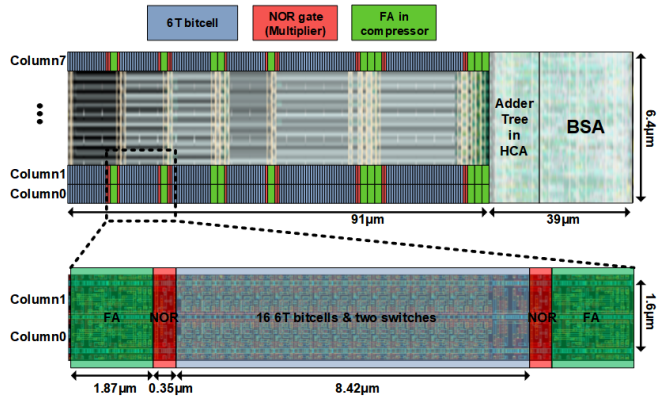


Fig. 17. Physical design of one column of the D6CIM.

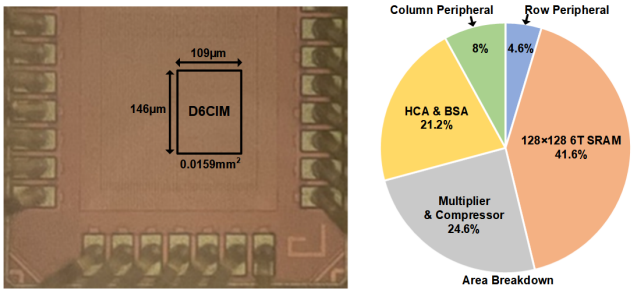


Fig. 18. Chip micrograph of the proposed D6CIM; and area breakdown.

the parasitic resistance and capacitance of these wires strongly impact throughput and energy efficiency. Fig. 17 shows the optimized layout of the eight columns and the corresponding HCA and BSA modules. The 16 bitcells and two NOR gates, which form one sub-column, are laid out tightly to reduce their combined horizontal length, since it reduces the LBL and LBLb, thereby increasing the robustness against read disturbance.

We also place the HCA and the BSA outside the sub-column array to reduce the WL and IL. We evaluated the alternative approach of inserting arithmetic hardware between the columns, as done in [25]; however, this can significantly increase the lengths of WLs and ILs. It also requires many wires between sub-columns and arithmetic hardware, which can disrupt the power-grid placement. The proposed physical design reduces the length of WLs and ILs by 30%.

Note that we place some of the FA cells of the compressors of the HCA near the NOR gates in sub-columns. See the green blocks in Fig. 17. The FA cell compresses a 3-bit input into a 2-bit output. Therefore, by placing the FA cells near the NOR gates, we can reduce the number of horizontal wires running toward the adder tree of the HCA. Also, note that we develop the layouts of the adder tree in the HCA and the BSA module using automatic place-and-route (APR) software. Therefore, it is hard to show the placement details for these blocks.

IV. Prototype and Measurements

A. Measurements

We prototype the D6CIM in a 28-nm CMOS technology. Fig. 18 shows the die photo. The macro takes a 0.0159-mm^2 active area. The 128 × 128.6T SRAM array occupies 41.6%

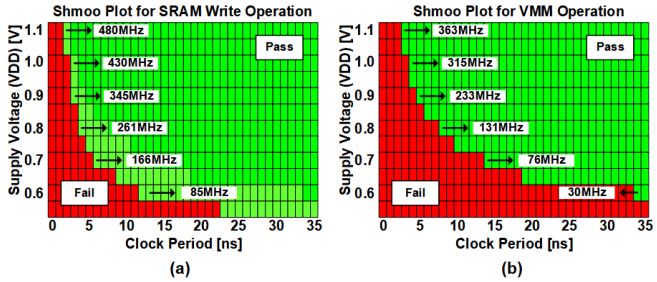


Fig. 19. Measured shmoo plot for (a) SRAM write operation and (b) VMM operation across supply voltages.

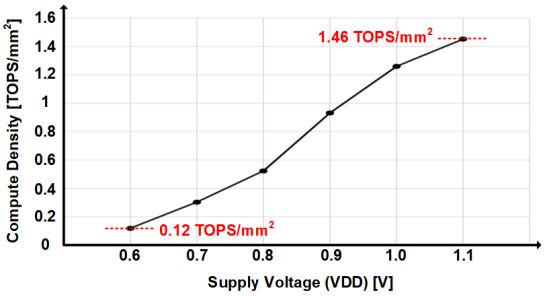


Fig. 20. Measured compute density (TOPS/mm^2) across supply voltages.

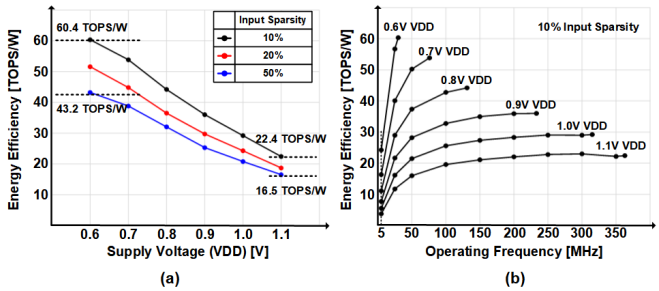


Fig. 21. Measured energy efficiency (TOPS/W) across (a) supply voltages and (b) operating frequencies.

of the total area, and the arithmetic hardware (including the multipliers, the HCAs, and the BSAs) takes 45.8%. A single bitcell has a footprint of 0.379 μm^2 in a logic rule. The peripheral hardware, which contains drivers and controllers, also occupies ~12.6% of the total area. It achieves a weight density of 1005 Kb/mm^2.

We fully characterize the macro's efficiency and throughput. Unless explicitly specified, we use 8-bit precision for inputs and weights, and 23-bit full precision for outputs. Fig. 19 shows the performance measurement. Fig. 19(a) shows the maximum operating frequencies across VDDs for SRAM write operation; Fig. 19(b) shows those for the VMM operation. At 1.1V, the maximum operating frequency is 480 MHz for write and 363 MHz for VMM. The macro achieves a compute density of 0.12-1.46 TOPS/mm^2 at 0.6-1.1V (Fig. 20). We measured the macro in 0.6-1.1V. The nominal voltage of this process technology is 0.9V.

Fig. 21 shows the energy efficiency measurement. Sweeping VDDs from 0.6-1.1V and input sparsity from 10% to 50%, the macro achieves the energy efficiency from 16.5 to 60.4 TOPS/W (Fig. 21(a)). For this test, we generate 50%

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:27:34 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON WITH PRIOR DIGITAL CIM DESIGNS

	This work	VLSI'25 [30]	JSSC'23 [27]	ISSCC'24 [16]	JSSC'24 [25]	ISSCC'22 [15]
Process [nm]	**28**	28	28	3	28	28
VMM operation	**Digital CIM**	Digital CIM	Digital CIM	Digital CIM	Digital CIM	Digital CIM
Bitcell type	**SRAM 6T**	SRAM -	SRAM -	SRAM 6T	SRAM 6T	SRAM 6T
Array size [b]	**16K**	96K	96K	60.75K	16K	32K
Supply voltage [V]	**0.6-1.1**	0.6-1.0	0.6-1.0	0.36-1.1	0.45-1.1	0.8
Frequency [MHz]	**30-360**	85-410	50-220	300-1600	250	333
Input precision [b]	**1-8**	4, 8	8, 16	12	1-4	1-8
Weight precision [b]	**8**	4	8, 16	12	1	1, 4, 8
Full-precision output	**Yes**	Yes	Yes	Yes	No	Yes
Signed/unsigned VMM support	**Yes**	Yes	Yes	Yes	No	No
Bitcell area [μm^2]	**0.379**	-	-	0.026	0.862	0.257
Macro area [mm^2]	**0.0159**	0.255	0.941	0.0157	0.049	0.03
Weight density [Kb/mm^2]	**1005**	376	102	3869	326	1067
[1)]Norm. weight density [Kb/mm^2]	**1005**	376	102	44.4	326	1067
[2)]Compute density [$TOPS/mm^2$]	**0.12-1.46**	2.47	1.43	23.8-123.75	2.59	0.17
[1,2)]Norm. compute density [$TOPS/mm^2$]	**0.12-1.46**	2.47	1.43	0.27-1.42	2.59	0.17
[2)]Energy efficiency [TOPS/W]	**22.4-60.4**	46.6	30.8-57.8	121	9.6-15.5	27.3
[3)]Area efficiency [$TOPS/mm^2 * Mb/mm^2$]	**0.12-1.43**	0.91	0.14	0.011-0.061	0.82	0.17
[4)]FoM [$TOPS/mm^2 * Mb/mm^2 * TOPS/W$]	**7.15-36.35**	[5)]23.1 @ 1V	4.38 @ 1V	1.41 @ 0.55V	7.91 @ 0.9V	4.83 @ 0.8V

[1)]Area is normalized quadratically to the 28-nm node.
[2)]One operation is defined as an 8-bit multiplication or addition.
[3)]Area efficiency is derived as a product of normalized weight density and normalized compute density.
[4)]FoM is derived as a product of normalized weight density, normalized compute density, and energy efficiency.
[5)]TOPS/W in 1V is estimated from chart.

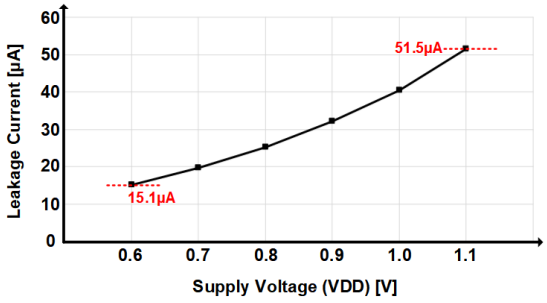


Fig. 22. Measured leakage current across supply voltages.

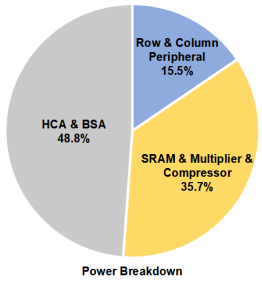


Fig. 23. Power breakdown for VMM operation. It is measured in 0.9V and 50% sparsity.

sparsity in a weight matrix. We also measure the energy efficiency across operating frequencies (Fig. 21(b)). On the other hand, Fig. 22 shows the leakage current measurement when the clock and input are gated to the macro. It consumes 15.1-51.5 μA at 0.6-1.1V.

Fig. 23 shows the energy consumption breakdown measurement of the in-memory computing operation. It shows that the HCA and the BSA account for ~48.8% of total energy consumption. The bitcells, multipliers, and compressors in the bitcell array consume 35.7%. The row and column peripherals consume the remaining 15.5%.

B. Comparisons

Table I presents the performance summary of the proposed D6CIM and the comparison to state-of-the-art DCIM prototypes. We include those prototypes that employ 8-bit integers and non-approximate arithmetic [30]. Many variable types of DCIM were presented indeed; however, we do not include those works that employ different configurations such as floating-point [31], embedded DRAM bitcells (eDRAM) [32], and hybrid digital-analog circuits [33] for a fair comparison. In the comparison, the D6CIM achieves the state-of-the-art weight density, only slightly less than [15]. It also achieves the third highest compute density of 1.46 TOPS/mm^2, preceded by [25], which achieves 2.59 TOPS/mm^2. The D6CIM achieves the second-best energy efficiency, second only to [16]. Note that one operation is either an 8-bit addition or a multiplication. To calculate the normalized weight density and compute density, we normalize the area to a 28-nm process technology. But we do not normalize it when calculating energy efficiency.

To more simply compare the state-of-the-art DCIM macros, we propose a FoM, which is the product of a normalized weight density (Mb/mm^2), a normalized compute density (TOPS/mm^2), and an energy efficiency (TOPS/W). Some prior works treat the product of TOPS/mm^2 and TOPS/W as a FoM [21]. Although these metrics can capture some of the merits of the DCIM macro design, we think those metrics can be artificially inflated. For example, we can increase the compute density by adding a large amount of arithmetic hardware. However, this does not compromise energy efficiency, as discussed in Sec. II, because the number of switchings/toggles,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:27:34 UTC from IEEE Xplore. Restrictions apply.

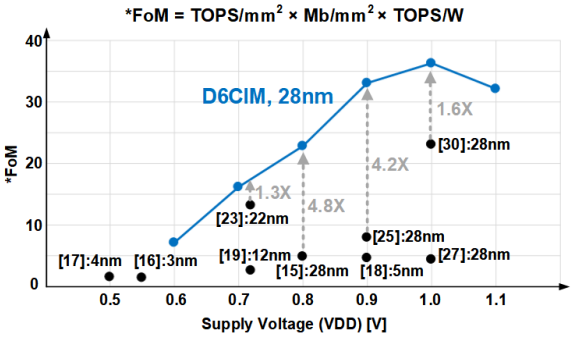


Fig. 24. Comparison with prior digital CIM designs in terms of FoM.

hence dynamic energy consumption, remains roughly the same across different degrees of hardware sharing. We can address this inflation problem by including Kb/mm^2 in the FoM, because adding a large amount of arithmetic hardware immediately reduces Kb/mm^2. Fig. 24 shows the comparisons in the proposed FoM. The D6CIM achieves the highest FoM across all VDDs, up to 4.8 times that of the existing macros.

V. Conclusion

This paper presents D6CIM, which can perform 1-to-8b VMM operations in a 128×128 SRAM array. We develop the D6CIM with both architecture- and circuit-level techniques. The D6CIM is prototyped in a 28-nm CMOS. The macro achieves a weight density of 1005 Kb/mm^2, a compute density of 1.46 TOPS/mm^2, and an energy efficiency of 60.4 TOPS/W, making D6CIM one of the state-of-the-art DCIM macros.

Appendix
Model Derivations

In this section, we present the process of developing the compute and weight density models of the D6CIM. We parameterize the models to simulate the macro with a range of circuit and architecture configurations. We begin our model development by formulating the number of cycles required for one VMM computation, N_{cycle}, as follows.

$$N_{cycle} = N_{in-bit} \cdot \frac{N_{row}}{N_{access} \cdot N_{subcol}}, \tag{7}$$

where N_{in-bit} is the bitwidth (precision) of the input, N_{row} is the number of rows of the bitcell array, N_{access} is the number of bitcell accesses each sub-column can make in each clock cycle, and N_{subcol} is the number of sub-columns in one column. Note that N_{in-bit} accounts for the bit serial scheme for the input that D6CIM adopts. N_{row}, N_{access}, N_{subcol} are 128, 2, 8, respectively, for D6CIM.

We can also formulate the computing throughput of the macro $TOPS$ as follows:

$$TOPS = \frac{2 \cdot N_{row} \cdot N_{col}}{N_{w-bit}} \cdot \frac{1}{N_{cycle}} \cdot f_{clk}, \tag{8}$$

where N_{col} is the number of columns of the bitcell array, N_{w-bit} is the bitwidth (precision) of weights, and f_{clk} is the clock frequency. The first term of the formula represents the total number of operations normalized to the bitwidth of weights. The operation is either addition or multiplication.

Then, we develop the area model of the macro A_{macro}, which can be formulated as below.

$$\begin{aligned} A_{macro} &= A_{mem} + A_{ari} + A_{peri} \\ &= A_{mem} + A_{ari} + k_{peri} \cdot (A_{mem} + A_{ari}) \\ &= (1 + k_{peri}) \cdot (A_{mem} + A_{ari}) \\ &= (1 + k_{peri}) \cdot (N_{row} \cdot N_{col} \cdot A_{6T} \\ &\quad + k_{ari} \cdot N_{ari} \cdot A_{FA}), \end{aligned} \tag{9}$$

where A_{peri} is the area of the peripheral circuits, A_{6T} is the area of a 6T bitcell, A_{FA} is the area of a FA cell, N_{ari} is the equivalent number of FA cells in an arithmetic logic unit. k_{peri} is the empirical parameter of the ratio of A_{peri} to $A_{mem} + A_{ari}$. Likewise, k_{ari} is the empirical parameter to estimate the area of non-FA cells (e.g., MUXes and inverters). we estimate k_{peri} to be 0.144 and k_{ari} to be 1.470, based on the layouts of D6CIM and other CIM macros.

N_{ari} is the key parameter in (9), which we can further elaborate as follows:

$$N_{ari} = N_{mult} + k_{HCA} \cdot N_{AT} + N_{SA}, \tag{10}$$

where N_{mult}, N_{AT}, and N_{SA} are the number of FA cells to implement the multipliers, adder-trees, and shift-accumulators in one macro, respectively. k_{HCA} is a hyperparameter that denotes the ratio of the HSA area to a conventional adder-tree area. k_{HCA} is estimated to be 0.85 for D6CIM.

D6CIM employs 1-bit multipliers in the sub-columns. Therefore, we can formulate N_{mult} as follows.

$$N_{mult} = \frac{A_{mult}}{A_{FA}} \cdot N_{col} \cdot N_{access} \cdot N_{subcol}, \tag{11}$$

where A_{mult} is the area of the 1-b multiplier. On the other hand, we can formulate N_{AT} and N_{SA} based on the conventional implementations [23]. The derivation of N_{AT} is as follows.

$$\begin{aligned} N_{AT} = &\sum_{i=1}^{log_2(N_{access} \cdot N_{subcol})} (N_{w-bit} + i - 1) \\ &\cdot \frac{(N_{access} \cdot N_{subcol})}{2^i} \cdot \frac{N_{col}}{N_{w-bit}} \end{aligned} \tag{12}$$

The derivation of N_{SA} is as follows.

$$\begin{aligned} N_{SA} = &(2 \cdot (log_2(N_{row}) + N_{w-bit} + N_{in-bit}) + N_{in-bit}) \\ &\cdot \frac{N_{col}}{N_{w-bit}} \end{aligned} \tag{13}$$

Note that we assume a ripple-carry adder architecture. Also, the shift-and-accumulator has accumulation registers. We assume the area of an N-bit register is the same as N FA cells.

Combining (8) and (9), we can formulate the compute density d_{comp} as follows.

$$\begin{aligned} d_{comp} &= \frac{TOPS}{A_{macro}} \\ &= \frac{2 \cdot N_{row} \cdot N_{col}}{N_{w-bit}} \cdot \frac{1}{N_{cycle}} \cdot f_{clk} \\ &\quad \cdot \frac{1}{(1 + k_{peri}) \cdot (A_{mem} + A_{ari})}. \end{aligned} \tag{14}$$

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:27:34 UTC from IEEE Xplore. Restrictions apply.

We also develop the weight density model d_{weight}, which can be formulated to:

$$d_{weight} = \frac{N_{row} \cdot N_{col}}{A_{macro}} = \frac{N_{row} \cdot N_{col}}{(1 + k_{peri}) \cdot (A_{mem} + A_{ari})}. \quad (15)$$

Using (14) and (15), we can predict d_{comp}, d_{weight}, and their product, across N_{subcol} and other parameters.

Additionally, we develop an energy efficiency model. We consider only the switching energy consumption, as it typically dominates the total energy consumption within the VDD range we consider. We can start to formulate the switching energy consumption as follows.

$$E_{sw} = k_{sw} \cdot C_{sw} \cdot {V_{DD}}^2 \cdot f_{clk}, \quad (16)$$

where k_{sw} denotes the switching probability and C_{sw} is the total switching capacitance. We can further develop the formula of C_{sw} as follows.

$$C_{sw} = k_{LBL} \cdot C_{LBL} \cdot N_{access} \cdot N_{subcol} \cdot N_{col} + C_{ari}, \quad (17)$$

$$k_{LBL} = \frac{1}{N_{in-bit}}, \quad (18)$$

where C_{LBL} is capacitance of one LBL and C_{ari} is capacitance of all logic of arithmetic hardware. k_{LBL} is a toggling ratio of LBL, which is related to the proposed bit-wise feeding scheme. We can simply express the energy efficiency(E_{energy}) as

$$E_{energy} = \frac{TOPS}{E_{sw}} = \frac{2 \cdot N_{row} \cdot N_{col}}{N_{w-bit}} \cdot \frac{1}{N_{cycle}} \cdot f_{clk} \cdot \frac{1}{k_{sw} \cdot C_{sw} \cdot {V_{DD}}^2 \cdot f_{clk}} = \frac{2 \cdot N_{row} \cdot N_{col}}{N_{w-bit}} \cdot \frac{1}{N_{cycle}} \cdot \frac{1}{k_{sw} \cdot C_{sw} \cdot {V_{DD}}^2}. \quad (19)$$

REFERENCES

[1] J.Song et al., "A 4-bit calibration-free computing-in-memory macro with 3T1C current-programed dynamic-cascode multi-level-cell eDRAM," *IEEE J. Solid-State Circuits*, vol. 59, no. 3, pp. 842–854, Mar.2024.

[2] M. E.Sinangil et al., "A 7-nm compute-in-memory SRAM macro supporting multi-bit input, weight and output and achieving 351 TOPS/W and 372.4 GOPS," *IEEE J. Solid-State Circuits*, vol. 56, no. 1, pp. 188–198, Jan.2021.

[3] P.Chen et al., "A 22-nm delta-sigma computing-in-memory SRAM macro with near-zero-mean outputs and LSB-first ADCs for edge AI processing," *IEEE J. Solid-State Circuits*, vol. 60, no. 8, pp. 3020–3032, Aug.2025.

[4] Z.Zhang et al., "A 28-nm 16-kb aggregation and combination computing-in-memory macro with dual-level sparsity modulation and sparse-tracking ADCs for GCNs," *IEEE J. Solid-State Circuits*, vol. 60, no. 3, pp. 949–962, Mar.2025.

[5] K.Lee, J.Kim, and J.Park, "A 28-nm 50.1-TOPS/W P-8T SRAM compute-in-memory macro design with BL charge-sharing-based in-SRAM DAC/ADC operations," *IEEE J. Solid-State Circuits*, vol. 59, no. 6, pp. 1926–1937, Jun.2024.

[6] H.Wang, R.Liu, R.Dorrance, D.Dasalukunte, D.Lake, and B.Carlton, "A charge domain SRAM compute-in-memory macro with C-2C ladder-based 8-bit MAC unit in 22-nm FinFET process for edge inference," *IEEE J. Solid-State Circuits*, vol. 58, no. 4, pp. 1037–1050, Apr.2023.

[7] X.Si et al., "A local computing cell and 6T SRAM-based computing-in-memory macro with 8-b MAC operation for edge AI chips," *IEEE J. Solid-State Circuits*, vol. 56, no. 9, pp. 2817–2831, Sep.2021.

[8] H.Diao et al., "A multiply-less approximate SRAM compute-in-memory macro for neural-network inference," *IEEE J. Solid-State Circuits*, vol. 60, no. 2, pp. 695–706, Feb.2025.

[9] Z.Jiang, S.Yin, J.-S.Seo, and M.Seok, "C3SRAM: An in-memory-computing SRAM macro based on robust capacitive coupling computing mechanism," *IEEE J. Solid-State Circuits*, vol. 55, no. 7, pp. 1888–1897, Jul.2020.

[10] Z.Chen et al., "CAP-RAM: A charge-domain in-memory computing 6T-SRAM for accurate and precision-programmable CNN inference," *IEEE J. Solid-State Circuits*, vol. 56, no. 6, pp. 1924–1935, Jun.2021.

[11] S.Xie, C.Ni, A.Sayal, P.Jain, F.Hamzaoglu, and J. P.Kulkarni, "EDRAM-CIM: Reconfigurable charge domain compute-in-memory design with embedded dynamic random access memory array realizing adaptive data converters," *IEEE J. Solid-State Circuits*, vol. 59, no. 6, pp. 1950–1961, Jun.2024.

[12] B.Zhang et al., "MACC-SRAM: A multistep accumulation capacitor-coupling in-memory computing SRAM macro for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 59, no. 6, pp. 1938–1949, Jun.2024.

[13] Z.Lin et al., "Two-direction in-memory computing based on 10T SRAM with horizontal and vertical decoupled read ports," *IEEE J. Solid-State Circuits*, vol. 56, no. 9, pp. 2832–2844, Sep.2021.

[14] S.Yin, Z.Jiang, J.-S.Seo, and M.Seok, "XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks," *IEEE J. Solid-State Circuits*, vol. 55, no. 6, pp. 1733–1743, Jun.2020.

[15] B.Yan et al., "A 1.041-Mb/mm^{2}27.38-TOPS/W signed-INT8 dynamic-logic-based ADC-less SRAM compute-in-memory macro in 28nm with reconfigurable bitwise operation for AI and embedded applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2022, pp. 188–190.

[16] H.Fujiwara et al., "34.4 A 3 nm, 32.5TOPS/W, 55.0TOPS/mm^{2}and 3.78Mb/mm^{2}fully-digital compute-in-memory macro supporting INT12 × INT12 with a parallel-MAC architecture and foundry 6T-SRAM bit cell," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 572–574.

[17] H.Mori et al., "A 4 nm 6163-TOPS/W/b **4790 – TOPS/mm^2/b** SRAM based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 132–134.

[18] H.Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm^{2}fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[19] C.-F.Lee et al., "A 12nm 121-TOPS/W 41.6-TOPS/mm^{2}all digital full precision SRAM-based compute-in-memory with configurable bit-width for AI edge applications," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 24–25.

[20] A.Guo et al., "A 28-nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," *IEEE J. Solid-State Circuits*, vol. 59, no. 9, pp. 3032–3044, Sep.2024.

[21] Y.He et al., "34.7 a 28nm 2.4Mb/mm^{2}6.9–16.3TOPS/mm^{2}eDRAM-LUT-based digital-computing-in-memory macro with in-memory encoding and refreshing," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 578–580.

[22] H.Zhang et al., "A 90.7-nW vibration-based condition monitoring chip featuring a digital compute-in-memory- based DNN accelerator using an ultra-low-power 13T-SRAM cell," *IEEE J. Solid-State Circuits*, vol. 60, no. 1, pp. 321–331, Jan.2025.

[23] Y.-D.Chih et al., "16.4 an 89TOPS/W and 16.3TOPS/mm^{2}all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 252–254.

[24] H.Kim, T.Yoo, T. T.-H.Kim, and B.Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, Jul.2021.

[25] C.-T.Lin et al., "DIMCA: An area-efficient digital in-memory computing macro featuring approximate arithmetic hardware in 28 nm," *IEEE J. Solid-State Circuits*, vol. 59, no. 3, pp. 960–971, Mar.2024.

[26] C.-T.Lin, P. X.Huang, J.Oh, D.Wang, and M.Seok, "IMCU: A 28-nm digital in-memory computing-based microcontroller unit for TinyML," *IEEE J. Solid-State Circuits*, vol. 59, no. 8, pp. 2684–2693, Aug.2024.

[27] F.Tu et al., "ReDCIM: Reconfigurable digital computing-in-memory processor with unified FP/INT pipeline for cloud AI acceleration," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 243–255, Jan.2023.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:27:34 UTC from IEEE Xplore. Restrictions apply.

[28] Y.Wang et al., "TensorCIM: Digital computing-in-memory tensor processor with multichip-module-based architecture for beyond-NN acceleration," *IEEE J. Solid-State Circuits*, vol. 60, no. 2, pp. 734–747, Feb.2025.
[29] F.Tu et al., "TranCIM: Full-digital bitline-transpose CIM-based sparse transformer accelerator with pipeline/parallel reconfigurable modes," *IEEE J. Solid-State Circuits*, vol. 58, no. 6, pp. 1798–1809, Jun.2023.
[30] Y.Wang et al., "LLM-CIM: A 28 nm 126.7TOPS/W input-LUT-based digital CIM macro with reconfigurable matrix multiplication and nonlinear operation modes for LLMs," in *Proc. Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2025, pp. 1–3.
[31] Y.Yuan et al., "14.5 A 28nm 192.3TFLOPS/W accurate/approximate dual-mode-transpose digital 6T-SRAM CIM macro for floating-point edge training and inference," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2025, pp. 258–260.
[32] J.Song et al., "A 1131-kb/mm^{2}14.0-to-53.3-TOPS/W 8-bit analog-assisted digital compute-in-memory with hybrid local-refresh eDRAM for attention computing," *IEEE J. Solid-State Circuits*, pp. 1–14, 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/11144478?casa_token=fbXQQ9vdMpcAAAAA:yU0gI1Yu60FE6_HVP24wQa9pUYhYT4W8VBzMz5J1F5_mfO5DEes_rEtVa8VMAWdk15B1nL0y
[33] Y.Zhao, Y.Wang, R. P.Martins, C.-H.Chan, S.Yin, and M.Zhang, "A hierarchical-hybrid floating-point compute-in-memory macro using FP-DAC and FP-ADC for edge-AI devices," *IEEE J. Solid-State Circuits*, pp. 1–14, 2025.
[34] J.Oh, C.-T.Lin, and M.Seok, "D6CIM: 60.4-TOPS/W, 1.46-TOPS/mm^2, 1005-Kb/mm^{2}digital 6T-SRAM-based compute-in-memory macro supporting 1-to-8b fixed-point arithmetic in 28-nm CMOS," in *Proc. IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2023, pp. 413–416.
[35] C.-T.Lin, S.Moon, P. X.Huang, and M.Seok, "EPIC: A sub-6 mW in-memory computing-based RISC-V microcontroller unit with on-chip training support for TinyML," in *Proc. IEEE Eur. Solid-State Electron. Res. Conf. (ESSERC)*, Sep. 2025, pp. 317–320.
[36] M.-F.Chang et al., "A compact-area low-VDDmin 6T SRAM with improvement in cell stability, read speed, and write margin using a dual-split-control-assist scheme," *IEEE J. Solid-State Circuits*, vol. 52, no. 9, pp. 2498–2514, Sep.2017.



Jonghyun Oh (Member, IEEE) received the B.S. and Ph.D. degrees in electrical and computer engineering from Seoul National University, Seoul, South Korea, in 2015 and 2021, respectively. In March 2021, he was a Post-Doctoral Researcher with the Inter-University Semiconductor Research Center, Seoul National University. In September 2021, he was a Post-Doctoral Researcher with Columbia University, New York, NY, USA. Since July 2023, he has been a Hardware Circuit Design Engineer with Apple, Cupertino, CA, USA. His main research interests include high-speed interfaces, integrated voltage regulators, synthesizable digital architectures, neuromorphic hardware accelerator design, and digital-type computing-in-memory macro design. He has served as a reviewer for various journals, including IEEE JOURNAL OF SOLID-STATE CIRCUITS, IEEE SOLID-STATE CIRCUITS LETTERS, IEEE ACCESS, and IEEE OPEN JOURNAL OF CIRCUITS AND SYSTEMS.



Chuan-Tung Lin (Member, IEEE) received the B.S. degree in electronics engineering from National Chiao Tung University, Hsinchu, Taiwan, in 2019, and the M.S. degree in electrical engineering from Columbia University, New York, NY, USA, in 2021, where he is currently pursuing the Ph.D. degree. His research interests include energy-efficient architecture for machine learning algorithms and VLSI circuit and system design.



Mingoo Seok (Senior Member, IEEE) received the B.S. degree (summa cum laude) in electrical engineering from Seoul National University, Seoul, South Korea, in 2005, and the M.S. and Ph.D. degrees in electrical engineering from the University of Michigan, Ann Arbor, MI, USA, in 2007 and 2011, respectively.

He was a member of the Technical Staff with Texas Instruments Inc., Dallas, TX, USA, in 2011. Since 2012, he has been with Columbia University, New York, NY, USA, where he is currently an Associate Professor of electrical engineering. His current research interests include ultra-low-power SoC design for emerging intelligent systems, machine-learning VLSI architecture and circuits, variation, voltage, aging, thermal-adaptive circuits and architecture, on-chip integrated power circuits, and nonconventional hardware design, including in-memory computing and analog-mixed-signal computing hardware. He is a member of the technical program committees for several conferences, including the IEEE International Solid-State Circuits Conference (ISSCC) and the ACM/IEEE Design Automation Conference (DAC). He received the 1999 Distinguished Undergraduate Scholarship from Korea Foundation for Advanced Studies, the 2005 Doctoral Fellowship from Korea Foundation for Advanced Studies, and the 2008 Rackham Pre-Doctoral Fellowship from the University of Michigan. He also received the 2009 AMD/CICC Scholarship Award for his work on the picowatt voltage reference and the 2009 DAC/ISSCC Design Contest for the 35-pW sensor platform design. He also received the 2015 NSF CAREER Award and the 2019 Qualcomm Faculty Award. He also received the Best Paper Award from IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION SYSTEMSin 2022. He serves/served as an Associate Editor for IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERSfrom 2013 to 2015, IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION SYSTEMSsince 2015, and IEEE SOLID-STATE CIRCUITS LETTERSsince 2017. He also served as the Guest Editor for IEEE JOURNAL OF SOLID-STATE CIRCUITS.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:27:34 UTC from IEEE Xplore. Restrictions apply.